GnarOne Game Math

Preset catalogue and customizability guide for the Rollerz SDK integration

2026-11-02 · build f751aa9 · https://danvid1009.github.io/gnarone-game-math

1. How presets work

Every engine is published as a browser module that implements the RGS provider contract and rolls locally. The SDK creates a game from a **preset name**, never from raw parameters:

```
import { createGame } from 'https://danvid1009.github.io/gnarone-game-math/<game>/api/rgs.js';
const g = createGame({ preset: 'MEDIUM' });
const s = g.open(); // session, balance, chip levels, preset tables
let r = g.bet({ sessionId: s.sessionId, betAmount: 1000, betType: 'MEDIUM' });
r = g.nextAction({ roundId: r.roundId, actionCode: 'CONTINUE' }); // multi-step games only
g.collect({ roundId: r.roundId });
```

A preset is a frozen configuration: the game parameters, the target RTP and the chip levels. Each preset is its own folder under **<game>/api/<preset>/** holding **field.json** (the config, the derived tables and the settlement rule), **rounds.json** and **rounds/NNN.json** (100 pre-drawn rounds from seeds fixture-000 to fixture-099, settled for every bet type) and a worked **sample-request / sample-response** pair. **index.json** at the game level lists presets, parameters and the realised RTP over the fixtures. Passing **seed** on a bet reproduces any fixture round through the live calls.

Money is always an integer in minor units. The bet response carries the balance minus the stake; collect adds the win. Every engine has exact RTP by construction for every bet type and, for the multi-step games, for every stopping rule; the Node test suites verify this by summation and by simulation.

Randomness and commitment

Every uniform an engine consumes is SHA-256(seed | stream | k) mapped to 53 bits, where the seed is 128 bits from the platform CSPRNG, minted per round and kept server-side. The bet response carries **seedHash** = SHA-256("commit" | seed), so the house is bound to the whole round before any further money (live bets) is placed; once the round ends every response carries **seed**, so the round can be recomputed and checked against the hash. The browser sandbox runs this same code client-side and therefore offers none of the secrecy; it is for integration, not money.

Three levels of customization

level	what changes	who does it	turnaround
Runtime	bet type and bet amount per round; which preset the client opens	client / SDK	none
Preset	a new row in a parameter table below (payout array, Elo field, top multiplier, RTP...)	GnarOne, via the preset config	same day: config, RTP verification, fixtures, publish
Structural	new mechanics or rules outside the parameter tables (payout structure, deck design, step rules)	GnarOne, ticket	design + math + tests

Anything not in a parameter table is structural. Anything in a table is a preset request: we add the row, verify the RTP exactly, generate the fixtures and publish it under its own folder.

Global parameters (every engine)

parameter	type	default	constraint
rtp	number	per engine	0 < rtp < 1; other constraints listed per engine
levels	integer[] minor units	100, 200, 500, 1000, 2500, 5000, 10000	non-empty; a bet must equal one of them
startingBalance	integer	100000	demo wallet in the sandbox only

2. Stepper

A fair crash ladder (RTP 0.97). Ten rungs; the player CONTINUEs or CASH_OUTs after each; one uniform drawn at bet time fixes where the ladder breaks. Survival to rung k is RTP / c_k, so the first step carries the whole house edge and every later step is a fair bet: every stopping rule returns exactly the RTP. Multi-step wire shape (CONTINUE, CASH_OUT, COLLECT).

Parameters

parameter	type	default	constraint
rtp	number	0.97	
maxWin	number	(preset)	greater than 1; sets c_k = round(maxWin$^{k/steps}$, 2)
steps	integer	10	1 to 20 sensible
returns	number[]	derived	alternative to maxWin/steps: strictly increasing, c_1 greater than rtp

Presets

preset	top win	rungs	rung 1 pays	P(instant crash)	P(reach the top)
EASY	10x	10	1.26x	23%	9.7%
MEDIUM	25x	10	1.38x	29.7%	3.9%
HARD	100x	10	1.58x	38.6%	1%

Who each preset is for

EASY	first-time players; the ladder is survivable and the top is reachable one round in ten
MEDIUM	the default; meaningful climbs with a 25x ceiling
HARD	high-variance players; every rung is a real risk, 100x once in a hundred rounds

Customizability

Runtime: the bet type is the preset name (the live Stepper sends difficulty as the bet type), so a client that offers three difficulties opens three games. Bet amount from the level list.

Preset: any top multiplier, any step count, any RTP. The rung schedule is derived, or an explicit list of rungs can be given instead (for example a slow start and a steep finish). The only constraint is that the first rung pays more than the RTP, otherwise the instant-crash probability would be negative.

Structural (ticket): anything that changes the survival rule, for example a non-zero payout on a crash, insurance, or a bonus rung. Those change the strategy-proofness argument and need a new derivation.

3. Single Shot

One draw, one payout (RTP 0.95). The unit interval is cut into bands whose widths are the probabilities; inside a band the paid multiple ramps linearly from a - r to a + r with r = 0.1 x the smallest gap between positive payouts, so the mean of each band is exactly its listed payout and the RTP is exact by summation. A zero band pays exactly 0. Single-call wire shape.

Parameters

parameter	type	default	constraint
rtp	number	0.95	strictly between the smallest and largest payout
payouts	number[]	(preset)	distinct, non-negative, at most one 0, at least two positive
method	geometric | maxent | weights	geometric	how probabilities are assigned to hit the RTP
weights	number[]	-	method weights only: relative frequencies of the winning payouts (the entry for 0 is ignored); the probability of the 0 payout is then computed so the RTP is exact
fraction	number	0.1	band half-width as a fraction of the smallest gap; fixed, not exposed

Presets

preset	payouts (x stake)	P(win)	stdev	band probabilities
classic	0, 0.5, 1, 2, 5, 20	51.1%	2.82x	0x 48.9% / 0.5x 25.5% / 1x 13.2% / 2x 6.9% / 5x 3.6% / 20x 1.9%
steady	0, 0.5, 1, 1.5, 2, 3	71.9%	0.9x	0x 28.1% / 0.5x 22% / 1x 17.3% / 1.5x 13.6% / 2x 10.6% / 3x 8.3%
spiky	0, 1, 2, 5, 20, 100	33.5%	5.65x	0x 66.5% / 1x 22.3% / 2x 7.5% / 5x 2.5% / 20x 0.8% / 100x 0.3%
always-pays	0.2, 0.5, 1, 2, 5	100%	1.25x	0.2x 39.5% / 0.5x 25.7% / 1x 16.8% / 2x 10.9% / 5x 7.1%
ragency	0, 0.25, 0.5, 0.8, 1, 1.2, 1.5, 2, 5, 10, 25, 49.98	93.8%	1.41x	0x 6.2% / 0.25x 5% / 0.5x 12% / 0.8x 23.6% / 1x 25% / 1.2x 17% / 1.5x 7% / 2x 3% / 5x 0.8% / 10x 0.3% / 25x 0.1% / 49.98x 0%

Who each preset is for

classic	the reference table; wins half the time, 20x top
steady	low variance; wins 72% of spins, nothing above 3x
spiky	high variance; wins a third of the time, rare 100x
always-pays	never returns zero; the top is capped at 5x

Customizability

Runtime: one bet type (BASE); only the amount varies.

Preset: the payout array itself is the input, so a new table is a preset row, not a design change: send the multiples you want and the RTP, we publish the folder. Geometric is the default probability method (each higher payout is a fixed factor rarer); maxent gives the flattest distribution that hits the RTP; weights lets you dictate the relative frequency of the winning payouts, and the engine then sets the probability of the 0 payout so the RTP is exact ($p_0 = 1 - RTP / E[payout \mid win]$). Any RTP between the smallest and largest payout works.
Direct input: because the RTP is exact by construction for any valid array, the SDK may also pass **payouts** (and optionally **method** and **weights**) straight to createGame instead of a preset name; the engine validates the array and throws on an invalid one. Presets exist for the fixtures and for reproducibility, not because other arrays are unsafe.

Structural (ticket): non-linear ramps inside a band, bands that overlap, or a payout that depends on the bet size.

4. Top 3 and Top 1

An Elo race (RTP 0.95). Strength is $10^{Elo/400}$; one uniform is recycled into the full finishing order (Plackett-Luce), so win, place and show probabilities are closed-form. Top 3 pays stake plus profit for first, a quarter of that profit for second and a fifth for third, with the profit solved per racer so every racer returns exactly the RTP. Top 1 pays the winner only at RTP / P(win). Single-call wire shape; the bet type is a placeholder RACER_1 .. RACER_n in Elo order, which the client reskins with its own names and art.

Parameters

parameter	type	default	constraint
rtp	number	0.95	
racers	elo[] (RACER_i placeholders)	(preset)	3 to 20 racers
mode	top3 | top1	top3	Top 1 is the same field paid winner-only
structure	{ place, show }	1/4, 1/5	Top 3 only: fractions of the win profit paid for 2nd and 3rd

Presets

preset	racers	Elo range	favourite wins	outsider wins	Top 3 multipliers, favourite	Top 1 odds fav / outsider
field-12	12	1850 to 1300	27%	1.1%	1.8x / 1.2x / 1.16x	3.52x / 83.53x
field-8	8	1800 to 1380	31.2%	2.8%	1.48x / 1.12x / 1.1x	3.05x / 34.19x
field-6	6	1750 to 1400	36.4%	4.9%	1.24x / 1.06x / 1.05x	2.61x / 19.57x
field-12-tight	12	1650 to 1452	13.8%	4.4%	3.86x / 1.71x / 1.57x	6.87x / 21.47x

Who each preset is for

field-12	the reference field: clear favourite, long shots at the back
field-8	fewer choices with a similar odds spread
field-6	short field for quick rounds; shortest prices
field-12-tight	near-even field; Top 1 odds cluster between 7x and 21x, Top 3 pays 3.9x for the favourite

Customizability

Runtime: the bet type is any RACER_i from the preset; the client can show the odds from open() before the first bet and maps RACER_i to its own skin.

Preset: any number of racers with any Elo values, so a new field is a preset row. For Top 3 we check one feasibility condition: the favourite's probability of finishing in the first three must stay below the RTP (field-12 sits at 66.4%, field-6 at 83.5%); a field with a very dominant favourite fails it and we tell you the largest Elo gap that works. The place and show fractions (1/4, 1/5) can be changed per preset. Top 1 has no feasibility condition.

Structural (ticket): exotic bets (exacta, trifecta), fixed-odds tables that do not return the RTP per racer, in-race cash-out, or racer-specific edges.

5. Rip and Rumble

Card packs from two decks (workbook Math-rippin-rumble-ver4). A pack is five cards drawn without replacement; each rarity pays count x value x multiplier(count), an exact hypergeometric sum. Each bet type draws from the low deck with the one probability that makes the expected win divided by the cost equal the RTP. Bundles give eleven packs for the price of ten. Single-call wire shape.

Parameters

parameter	type	default	constraint
rtp	number	0.95	must lie between the two decks' own returns; the low/high mix is solved
credit	integer minor units	100	price of one credit; a BASE pack costs 5 credits
betTypes	table	BASE 5 / BOOSTED 10 / BASE_BUNDLE 5x10 (11 packs) / BOOSTED_BUNDLE 10x10 (11)	cost in credits, packs dealt, packs paid
decks, pays, multipliers	workbook tables	ver4	structural

Presets

preset	RTP	P(low deck) BASE	BOOSTED	BASE_BUNDLE	BOOSTED_BUNDLE	stdev (x stake) BASE / BOOSTED
sheet-v4	0.95	87.3%	8.2%	94.5%	22.6%	1.62 / 1.49
sheet-v4-0.93	0.93	89%	11.6%	96%	25.6%	1.58 / 1.47
sheet-v4-0.97	0.97	85.7%	4.9%	93%	19.6%	1.67 / 1.51

Who each preset is for

sheet-v4	the reference sheet at 0.95
sheet-v4-0.93	tighter margin; same feel
sheet-v4-0.97	looser margin; same feel

Customizability

Runtime: four bet types; the amount must be a multiple of the pack price (cost x paid packs x credit), so BASE bets are 500, 1000, 2500, 5000 minor units and BASE_BUNDLE 5000, 10000, 25000, 50000.

Preset: the RTP (we re-solve the deck mix, which is why 0.93 and 0.97 are presets), the credit price, and the bet-type table (pack costs, bundle sizes). Note that BOOSTED_BUNDLE is not in the current live SDK bet-type list; drop it from the preset or add it to the SDK.

Structural (ticket): deck composition, per-rarity pays, the count multipliers, pack size, or a third deck. Each changes the hypergeometric tables and needs re-verification against a new workbook.

6. Game of Three

Rock n Smash (workbook Rock n Smash.xlsx). Three rocks; a fixed table gives how many break (0 to 3) and pays 5 / 1 / 0.5 / 0; on two or three breaks a gem fires with a fixed chance and multiplies the win by 2x to 10x from a weighted table (mean 3.667). The RTP is the exact sum over 16 outcome zones. Single-call wire shape.

Parameters

parameter	type	default	constraint
rtp	number	as sheet	if set, the win table is scaled so both modes return it
modes	rows (count probabilities, gem chance, wins)	BASE, BOOSTED from the workbook	structural
gems	weighted table 2x to 10x	workbook	structural

Presets

preset	BASE RTP	BASE hit rate	BASE stdev	BOOSTED RTP	BOOSTED hit rate	BOOSTED stdev
sheet	97.07%	76%	1.84x	97.15%	88%	1.6x
rtp-0.95	95%	76%	1.8x	95%	88%	1.56x

Who each preset is for

sheet	exact workbook numbers, 97.07% and 97.15%
rtp-0.95	the same game at a 95% return

Customizability

Runtime: two bet types, BASE and BOOSTED; BOOSTED is the SDK's boosted bet type.

Preset: the RTP target. We keep the workbook's hit probabilities and gem chances and scale the win amounts, so the feel of the game is unchanged and only the payouts move.

Structural (ticket): different break probabilities, gem odds or the gem multiplier table, more rocks, or a separate bonus round.

7. Pick One

The general RTP / p engine (RTP 0.95). Give it any probability vector $p_1..p_n$ (or weights, or just n for equal options); option i owns a zone of width p_i and pays RTP / p_i on a hit, so every option returns exactly the RTP whatever the vector. Options are placeholders OPTION_1 .. OPTION_n that the client reskins (coin faces, doors, colours). Top 1 is this same formula with p from Elo. Single-call wire shape.

Parameters

parameter	type	default	constraint
rtp	number	0.95	
n	integer	(preset)	at least 2 equal options
weights	number[]	-	alternative to n; positive, any scale
probabilities	number[]	-	alternative to n; must sum to 1

Presets

preset	options (probability, odds)
coin	OPTION_1 (50%, 1.9x), OPTION_2 (50%, 1.9x)
three	OPTION_1 (33.3%, 2.85x), OPTION_2 (33.3%, 2.85x), OPTION_3 (33.3%, 2.85x)
five	OPTION_1 (20%, 4.75x), OPTION_2 (20%, 4.75x), OPTION_3 (20%, 4.75x), OPTION_4 (20%, 4.75x), OPTION_5 (20%, 4.75x)
weighted	OPTION_1 (50%, 1.9x), OPTION_2 (30%, 3.17x), OPTION_3 (20%, 4.75x)

Who each preset is for

coin	two equal options at 1.9x (skin: a coin flip)
three	three equal options at 2.85x
five	five equal options at 4.75x
weighted	unequal options 5:3:2, paying 1.9x / 3.17x / 4.75x

Customizability

Runtime: the bet type is any option name; the amount from the level list.

Preset: any number of options, equal or weighted, any RTP. This is the most freely customizable engine: a preset is literally the probability vector.
Direct input: as with Single Shot, the SDK may pass **n**, **weights** or **probabilities** directly to createGame; the odds are exact for any input.

Structural (ticket): multi-pick bets, partial payouts for near misses, or options whose probability depends on earlier rounds.

8. Locker Wars

An ascending locker auction played as raises (RTP 0.95). Raise k is its own bet of b_1 x m_k, where the auctioneer adds an increment drawn from the stage's set (storage: +1, then +1 or +2, +1 or +2, +2 or +3, +2 or +4 opening bids), so the ladder varies between auctions and every bid is a round number. With probability H_k the rivals outbid you and the bid comes back in full; otherwise they fold, you win the locker and open it: a Single Shot draw on the listed multiples whose mean, (R - H_k) / (1 - H_k), is solved so that the raise returns exactly R with the refund included. RTP is therefore fixed at R per unit staked for every raise and for every raise-or-walk policy; the locker mean is lowest where outbids are most frequent and equals R at the last stage. edge: 'hammer' is available instead to give every won locker mean R (a buyer's premium), in which case the return per unit staked rises above R. The last stage has H = 0: the rivals are gone. Single-call wire shape: bet type BID_k, response carries outcome, the locker on a win, and the next raise (next is null on a win).

Parameters

parameter	type	default	constraint
rtp	number	0.95	
refund	number	1	fraction of the bid returned on an outbid
edge	per-raise | hammer	per-raise	per-raise: every raise returns R, locker mean (R - H) / (1 - H); hammer: won lockers have mean R
outbid	number[]	(preset)	P(outbid) per raise, each in [0,1); the last should be 0
steps	number[][]	(preset)	allowed increments between consecutive raises, in opening-bid units, drawn uniformly per raise; BID_k levels = chip levels x every multiple the stage can reach
bids	number[]	-	alternative: a fixed ladder of multiples
locker.payouts	number[]	(preset)	locker multiples, 0 allowed (empty); mean solved per stage, must be positive
locker.method	weights | maxent | geometric	weights	weights: the positive multiples keep a fixed shape and only P(empty) moves with the mean
locker.weights	number[]	(preset)	relative frequency of each positive multiple (a smooth hump 0.5 to 3x, thin to the top)

Presets

preset	raises	increments per raise	P(outbid) per raise	locker multiples	locker mean per stage	E[hammer]	RTP	largest win
street	4	1 · 1/2 · 1/2	0.5, 0.35, 0.2, 0	0, 0.5, 0.8, 1.2, 1.5, 2, 3, 5	0.9, 0.923, 0.938, 0.95	1.81x b1	95%	30x b1
storage	6	1 · 1/2 · 1/2 · 2/3 · 2/4	0.6, 0.5, 0.4, 0.3, 0.2, 0	0, 0.5, 0.8, 1.2, 1.5, 2, 3, 5, 7	0.875, 0.9, 0.917, 0.929, 0.938, 0.95	2.34x b1	95%	91x b1
vault	7	1 · 1/2 · 2/3 · 2/3/5 · 3/5 · 4/6/8	0.7, 0.6, 0.5, 0.4, 0.3, 0.2, 0	0, 0.5, 0.8, 1.2, 1.5, 2, 3, 5, 7, 10	0.833, 0.875, 0.9, 0.917, 0.929, 0.938, 0.95	3.27x b1	95%	250x b1

Who each preset is for

street	four quick raises, top bid 4 to 6x the opener, 5x top locker
storage	six raises, top bid 8 to 13x, the reference auction, 7x top locker
vault	seven raises, top bid 14 to 25x, 10x top locker, slowest to win

Customizability

Runtime: the bet type is the raise number (BID_1 .. BID_N) and the amount its level; the client chains raises with the response's next field. Walking away is simply not placing the next raise.

Preset: the outbid schedule, the increment sets, the locker multiples, the refund fraction and the edge rule are all preset rows; the increment sets are checked so every amount names its opener unambiguously. With the default edge rule the quoted RTP is a per-preset figure for raise-until-won (above R, since refunded stakes count as staked); per-raise mode pins every raise at R if compliance needs a single number.

Structural (ticket): rivals whose behaviour depends on the player's history, a locker drawn once at the start and revealed later (changes the information structure), or live side markets on the auction.

9. Live Fight

Two fighters, a biased damage walk, a health bar each (RTP 0.95). Each tick one fighter lands a hit, with probability w / (1 + w), $w = 10^{(powerA - powerB)/1200}$; damage is half-normal with mean 16 x $w^{+/-0.25}$ on a 0.5 HP grid. First to zero loses; if both stand after the last tick a biased coin decides on the health share. P(A wins | tick, healths) is exact by backward induction on the grid and is a martingale, so the opening bet is priced at RTP / P and every live bet during the fight is a fair bet at 1 / P. Multi-step wire shape (CONTINUE reveals one tick) plus linked bets for live money.

Parameters

parameter	type	default	constraint
rtp	number	0.95	opening bets only; live bets are rakeless
A, B	{ power, hp }	(preset)	power on an Elo-like scale, hp greater than 0
ticks	integer	16	length of the fight before the decision coin
baseDamage	number	16	mean damage at equal power
damageExponent	number	0.25	how much power leans on damage size (0 = frequency only)
eloScale	number	1200	power gap for a 10x strength ratio
res	number	0.5	health grid resolution; damage is rounded to it
endRule	health-share | coin-half	health-share	decision rule when both stand at the bell

Presets

preset	A (power / hp)	B (power / hp)	ticks	A lands	mean dmg A / B	P(A wins)	odds A / B	KO rate
even	1600 / 100	1600 / 100	16	50%	16 / 16	50%	1.9x / 1.9x	96.9%
favourite	1700 / 100	1550 / 100	16	57.1%	17.2 / 14.9	72.9%	1.3x / 3.5x	97.4%
glass-cannon	1700 / 70	1550 / 120	16	57.1%	17.2 / 14.9	45.1%	2.11x / 1.73x	98.4%
slugfest	1600 / 100	1600 / 100	10	50%	24 / 24	50%	1.9x / 1.9x	92%

Who each preset is for

even	equal fighters, 50/50 at the bell, the widest live swings
favourite	a 150-point gap: 73% favourite, 1.29x against 3.59x
glass-cannon	more power but less health: rates 45%, shows HP and power trading off
slugfest	ten heavy-hitting ticks: short fights, big swings per hit

Customizability

Runtime: back A or B at the bell at the opening odds; during the fight, quote() returns the fair live odds for the public state and linkedBet() places new money at them, settled when the fight ends. Live betting is optional for the client.

Preset: any pairing of power and HP, the tick count, base damage, the power-to-damage exponent, the Elo scale and the decision rule. The rating P(A wins) is recomputed exactly for any combination, so a new matchup is a preset row, not a design change.

Structural (ticket): hits that heal, multi-fighter brawls, props other than the winner (next hit lands, KO within k ticks) which are exact from the same grid but need their own pricing calls, or paid live bets (an edge on live money).

10. Cutscene Fight

One bet on the winner, decided by a single biased coin whose probability is the Live Fight rating of the pairing, then a predetermined cutscene that plays it out (RTP 0.95). During the cutscene the developer may call any number of side bets from a library; each is an independent draw at its own RTP. Single-call wire shape plus sideBet().

Parameters

parameter	type	default	constraint
rtp	number	0.95	main bet
A, B	{ power, hp }	(preset)	rated with the Live Fight grid (same defaults)
fight	Live Fight options	defaults	ticks, damage, scale, used for the rating and the cutscene
side bet: coin	{ p, pick, rtp }	p 0.5	OPTION_1 / OPTION_2 pay rtp / p and rtp / (1 - p)
side bet: pick	{ n | weights | probabilities, pick, rtp }	-	OPTION_i pays rtp / p_i
side bet: single-shot	{ payouts, method, weights, rtp }	-	sloped-band table, exact rtp

Presets

preset	A (power / hp)	B (power / hp)	P(A wins)	odds A / B
even	1600 / 100	1600 / 100	50%	1.9x / 1.9x
favourite	1700 / 100	1550 / 100	72.9%	1.3x / 3.5x
glass-cannon	1700 / 70	1550 / 120	45.1%	2.11x / 1.73x
side-bet library				
coin	coin	{"p": 0.5}		rtp 0.95
biased-coin	coin	{"p": 0.7}		rtp 0.95
pick-3	pick	{"n": 3}		rtp 0.95
pick-weighted	pick	{"weights": [5, 3, 2]}		rtp 0.95
shot-classic	single-shot	{"payouts": [0, 0.5, 1, 2, 5, 20]}		rtp 0.95

Who each preset is for

even	50/50 coin, 1.90x each side
favourite	73% favourite
glass-cannon	power against health, 45%

Customizability

Runtime: back A or B; call sideBet() with a library name or raw parameters as often as the game wants, before, during or after the cutscene. Options are placeholders OPTION_1..n.

Preset: any pairing (rated exactly), any entries in the side-bet library: a coin probability, a pick vector, a payout array, each with its own RTP.

Structural (ticket): side bets that depend on the fight (who lands next, round of the KO), or a cutscene that is not purely cosmetic.

11. Distributions

A library of sixteen named distributions behind one uniform: a sample is the inverse CDF of a seeded draw, reproducible and auditable. Free sealed draws for anything a game needs random (bot speeds, timings, Ragency targets), and a money mode where the stake pays a multiple sampled from a chosen distribution, clamped to [min, max] and scaled so its mean is exactly the RTP. Single-call wire shape plus draw().

Parameters

parameter	type	default	constraint
dist	library name	(preset)	any entry except permutation for money mode
params	per distribution	library defaults	validated per entry (see table)
min	number	0	floor on the raw sample, >= 0
max	number	none	cap on the raw sample; required when the raw mean is infinite
rtp	number	0.95	E[multiple] exactly; the scale c = rtp / E[clamped sample]

Presets

preset	distribution	parameters	clamp	scale c	max multiple	P(win > 0)
lognormal	lognormal	{"mu": 0, "sigma": 0.8}	[0, 50]	0.6898	34.49x	100%
exponential	exponential	{"rate": 1}	[0, 20]	0.95	19x	100%
uniform	uniform	{"a": 0, "b": 1}	[0, none]	1.9	1.9x	100%
normal-clipped	normal	{"mu": 1, "sigma": 0.5}	[0, 3]	0.946	2.84x	97.7%
pareto	pareto	{"xm": 0.5, "alpha": 2.5}	[0, 100]	1.1402	114.02x	100%
poisson	poisson	{"lambda": 1}	[0, none]	0.95	13.3x	63.2%
gamma	gamma	{"k": 2, "theta": 0.5}	[0, 20]	0.95	19x	100%
loglogistic	loglogistic	{"alpha": 1, "beta": 3}	[0, 50]	0.7858	39.29x	100%

Customizability

Runtime: draw() with any library entry and parameters, as often as needed, no money; bet on the preset's money distribution.

Preset: any distribution, parameters, floor, cap and RTP for money mode; the scale is solved and the RTP verified by summation. The library itself can grow (gamma, beta, mixtures) as preset-level work since each entry is one quantile function.

Structural (ticket): payouts that depend on more than one draw, correlated draws, or a multiple that depends on the bet size.

Library

entry	kind	parameters (defaults)
uniform	continuous	a=0, b=1
triangular	continuous	a=0, b=1, c=0.5
normal	continuous	mu=0, sigma=1
lognormal	continuous	mu=0, sigma=0.5
halfnormal	continuous	sigma=1
exponential	continuous	rate=1
logistic	continuous	mu=0, s=1
weibull	continuous	k=1.5, lambda=1
pareto	continuous	xm=1, alpha=2
gamma	continuous	k=2, theta=1
chisquare	continuous	df=3

entry	kind	parameters (defaults)
loglogistic	continuous	alpha=1, beta=3
rayleigh	continuous	sigma=1
frechet	continuous	alpha=3, s=1, m=0
bernoulli	discrete	p=0.5
binomial	discrete	n=10, p=0.5
poisson	discrete	lambda=3
geometric	discrete	p=0.3
categorical	discrete	weights=[1,1]
dice	discrete	sides=6
permutation	discrete	n=5

Appendix A: race fields

Every racer in every Top 3 / Top 1 preset, with the closed-form win probability and the multipliers each mode pays (Top 3: first / second / third; Top 1: winner).

field-12 (12 racers, Elo 1850 to 1300)

racer	Elo	P(win)	Top 1 pays	Top 3 pays 1st / 2nd / 3rd
RACER_1	1850	27%	3.52x	1.8x / 1.2x / 1.16x
RACER_2	1780	18%	5.27x	2.7x / 1.42x / 1.34x
RACER_3	1720	12.8%	7.44x	3.95x / 1.74x / 1.59x
RACER_4	1680	10.1%	9.37x	5.11x / 2.03x / 1.82x
RACER_5	1640	8.1%	11.8x	6.6x / 2.4x / 2.12x
RACER_6	1600	6.4%	14.85x	8.5x / 2.87x / 2.5x
RACER_7	1560	5.1%	18.7x	10.9x / 3.48x / 2.98x
RACER_8	1520	4%	23.54x	13.94x / 4.24x / 3.59x
RACER_9	1480	3.2%	29.64x	17.78x / 5.19x / 4.36x
RACER_10	1430	2.4%	39.52x	24.01x / 6.75x / 5.6x
RACER_11	1380	1.8%	52.71x	32.32x / 8.83x / 7.26x
RACER_12	1300	1.1%	83.53x	51.78x / 13.69x / 11.16x

field-8 (8 racers, Elo 1800 to 1380)

racer	Elo	P(win)	Top 1 pays	Top 3 pays 1st / 2nd / 3rd
RACER_1	1800	31.2%	3.05x	1.48x / 1.12x / 1.1x
RACER_2	1740	22.1%	4.3x	2.02x / 1.25x / 1.2x
RACER_3	1680	15.6%	6.08x	2.92x / 1.48x / 1.38x
RACER_4	1620	11.1%	8.59x	4.34x / 1.84x / 1.67x
RACER_5	1560	7.8%	12.13x	6.43x / 2.36x / 2.09x
RACER_6	1500	5.5%	17.14x	9.42x / 3.11x / 2.68x
RACER_7	1440	3.9%	24.21x	13.69x / 4.17x / 3.54x
RACER_8	1380	2.8%	34.19x	19.74x / 5.69x / 4.75x

field-6 (6 racers, Elo 1750 to 1400)

racer	Elo	P(win)	Top 1 pays	Top 3 pays 1st / 2nd / 3rd
RACER_1	1750	36.4%	2.61x	1.24x / 1.06x / 1.05x
RACER_2	1680	24.3%	3.9x	1.68x / 1.17x / 1.14x
RACER_3	1610	16.3%	5.84x	2.55x / 1.39x / 1.31x
RACER_4	1540	10.9%	8.74x	4.09x / 1.77x / 1.62x
RACER_5	1470	7.3%	13.08x	6.53x / 2.38x / 2.11x
RACER_6	1400	4.9%	19.57x	10.25x / 3.31x / 2.85x

field-12-tight (12 racers, Elo 1650 to 1452)

racer	Elo	P(win)	Top 1 pays	Top 3 pays 1st / 2nd / 3rd
RACER_1	1650	13.8%	6.87x	3.86x / 1.71x / 1.57x
RACER_2	1632	12.5%	7.62x	4.32x / 1.83x / 1.66x
RACER_3	1614	11.2%	8.45x	4.85x / 1.96x / 1.77x
RACER_4	1596	10.1%	9.37x	5.43x / 2.11x / 1.89x
RACER_5	1578	9.1%	10.4x	6.08x / 2.27x / 2.02x
RACER_6	1560	8.2%	11.53x	6.81x / 2.45x / 2.16x

racer	Elo	P(win)	Top 1 pays	Top 3 pays 1st / 2nd / 3rd
RACER_7	1542	7.4%	12.79x	7.63x / 2.66x / 2.33x
RACER_8	1524	6.7%	14.19x	8.53x / 2.88x / 2.51x
RACER_9	1506	6%	15.73x	9.53x / 3.13x / 2.71x
RACER_10	1488	5.4%	17.45x	10.65x / 3.41x / 2.93x
RACER_11	1470	4.9%	19.36x	11.89x / 3.72x / 3.18x
RACER_12	1452	4.4%	21.47x	13.27x / 4.07x / 3.45x

Appendix B: files per game

```
https://danvid1009.github.io/gnarone-game-math/<game>/api/rgs.js — the module (createGame)
https://danvid1009.github.io/gnarone-game-math/<game>/api/index.json — presets, parameters, bet types, realised RTP
https://danvid1009.github.io/gnarone-game-math/<game>/api/<preset>/field.json — frozen config, derived tables, how_to_settle
https://danvid1009.github.io/gnarone-game-math/<game>/api/<preset>/rounds.json, rounds/NNN.json — 100 seeded rounds settled for every bet type
https://danvid1009.github.io/gnarone-game-math/<game>/api/<preset>/sample-request.json, sample-response.json — round 007 through the real calls
```

Games: stepper, single-shot, top-3, top-1, rip-rumble, game-of-three, pick-one, auction, fight, cutscene, distributions. Each page also carries the full algorithm write-up, figures, and a playable demo. Live Fight adds quote() and linkedBet(); Cutscene Fight adds sideBet().